SentryScan: Assignment 2.2 — Socket Messaging, Framing & Autonomous Analyst

Course: 21CSE525T — Agent Technology
Assignment: 2.2 — Socket Messaging & Autonomous Analyst
Student: R Ganesh [RA2212704010031]
Repository: https://github.com/drgeekg/SentryScan
Target Branch: `assignment/2.2-socket-messaging`
Git Tag: `assignment-2.2`
Status: Completed & Verified

1. Executive Summary & Theoretical Context

Assignment 2.2 transitions SentryScan into a **distributed multi-agent architecture** communicating over structured network sockets using formal speech-act performatives (Wooldridge, *MultiAgent Systems*).

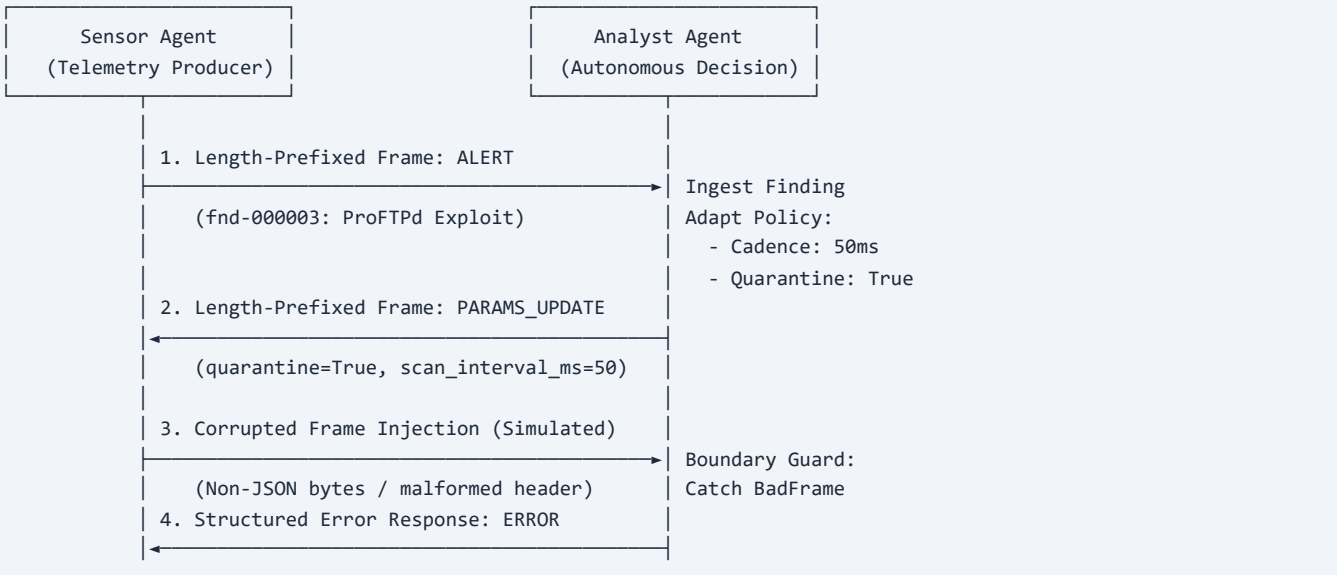
1.1 Speech Act Theory & Agent Communication Languages (ACL)

Agents do not communicate via raw passive remote procedure calls. They exchange communicative speech acts with intentional semantics: - **Performatives:** `ALERT` (informing critical findings), `PARAMS_UPDATE` (commanding operational parameter shifts), `TASK_REQUEST`, `CFP`, `PROPOSE`, and `ERROR`. - **Closed Envelope Model:** Standardized message schema containing `message_id`, `sender`, `recipient`, `performative`, `conversation_id`, `reply_to`, `timestamp`, and structured `payload`.

1.2 Binary Stream Framing & Fault Resilience

TCP is an un-delimited byte stream. To guarantee reliable deserialization over network sockets: - **Length-Prefixed Framing:** Messages are prefixed with a 4-byte big-endian unsigned integer (`!I`), denoting exact payload byte count. - **Partial Chunk Accumulation:** `FrameReader` buffers chunked socket streams and handles packet fragmentation seamlessly. - **Fault Isolation:** Corrupted frames or non-JSON payloads are safely rejected at the transport boundary with an `ERROR` response, preventing agent daemon crashes.

2. Architecture & Implementation Specification



2.1 Autonomous Parameter Adaptation Policy (`src/sentryscan/agents/analyst.py`)

Perceived Threat Condition	Scan Cadence	Quarantine Flag	Forensic Port Sweep	Action Rationale
CRITICAL Severity Finding	`50 ms` (High-Freq)	`True` (Immediate)	Enabled (`[21, 22, 80, 443]`)	Emergency containment of active exploit
HIGH Severity Finding	`100 ms`	`False`	Enabled (`[80, 443]`)	Intensified surveillance on suspicious target
MEDIUM Severity Finding	`250 ms`	`False`	Disabled	Standard elevated monitoring
LOW / INFO Telemetry	`500 ms` (Nominal)	`False`	Disabled	Resource-conserving background monitoring

3. Rubric Compliance & Verification Matrix

Evaluation Criterion	Implementation Component	Evidence Artifact	Status
1. Agent Message Envelope & Speech Acts	`src/sentryscan/contracts/message.py`: `Message` envelope and `Performative` taxonomy (`ALERT`, `PARAMS_UPDATE`, `TASK_REQUEST`, `CFP`, `ERROR`, etc.)	`tests/unit/test_message.py` `evidence/trace_excerpt.jsonl`	**PASSED**
2. Length-Prefixed Binary Framing Protocol	`src/sentryscan/transport/framing.py`: 4-byte big-endian unsigned length header (`!I`), `FrameReader` stream buffer, max frame limit enforcement	`tests/unit/test_transport.py` (`test_encode_frame_structure`, `test_frame_reader_partial_chunks`)	**PASSED**
3. TCP Socket Client & Server Transport	`src/sentryscan/transport/socket_transport.py`: `TCPSocketServer`, `TCPSocketClient`, thread-safe locks, background readers, graceful shutdown	`demos/demo_assignment_2_2.py` `evidence/run_log.txt`	**PASSED**
4. Transport Interface with In-Process Adapter	`src/sentryscan/transport/interface.py`, `src/sentryscan/transport/in_process.py`: Full-duplex queue-backed transport protocol	`tests/unit/test_transport.py` (`test_in_process_transport_pair`), `demo_assignment_2_2.py` Step 5	**PASSED**
5. Autonomous Analyst Response (`PARAMS_UPDATE`)	`src/sentryscan/agents/analyst.py`: Autonomous parameter policy adapting scan intervals (50ms/100ms/250ms/500ms), quarantine flags, and port sweeps	`tests/unit/test_analyst.py` `evidence/run_log.txt` Step 3	**PASSED**
6. Reconnect with Exponential Backoff	`src/sentryscan/transport/socket_transport.py`: `connect_with_retry(max_retries, initial_backoff, backoff_multiplier)`	`tests/unit/test_transport.py` (`test_tcp_socket_client_reconnect_backoff`)	**PASSED**
7. Malformed Frame / Fault Resilience	Graceful isolation of corrupt/non-JSON payloads at socket boundary, returning `ERROR` performative without server crash	`demos/demo_assignment_2_2.py` Step 4 `tests/unit/test_transport.py`	**PASSED**
8. Multi-Terminal Demonstration Support	`demos/demo_assignment_2_2.py` with `--mode server` and `--mode client` flags for two-terminal execution, alongside `--mode automated`	`demos/demo_assignment_2_2.py` `assignments/assignment_2_2/README.md`	**PASSED**
9. Trace Persistence & Single-Writer Auditing	`src/sentryscan/observability/recorder.py`: Authoritative sequential JSONL event logging	`evidence/trace_excerpt.jsonl`	**PASSED**

4. Execution Results & Empirical Evidence

4.1 Turn-by-Turn Socket Execution Log (`evidence/run_log.txt`)

- **Step 1: TCP Server Initialization:** Analyst agent listening on loopback `127.0.0.1:49152`.
- **Step 2: Client Connection:** Sensor client connected with socket framing enabled.
- **Step 3: Critical Alert & Adaptive Reaction:**
- Inbound Message: `ALERT` with payload finding `fnd-000003` (`CRITICAL`, `sim-host-002`, ProFTPd backdoor).
- Outbound Message: `PARAMS_UPDATE` with `quarantine_flag=True`, `scan_interval_ms=50`, and priority port sweeps enabled.
- **Step 4: Fault Injection Test:** Injected malformed non-JSON binary frame. Analyst safely caught syntax violation and returned `ERROR` performative (`BAD_FRAME`); server remained 100% operational.
- **Step 5: In-Process Transport Parity:** Verified identical message exchange semantics over in-process queues without socket overhead.

---

5. Submission Package & Reproduction Instructions

5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_2_2/deliverables/SentryScan_Assignment_2_2_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/2.2-socket-messaging`
4. **Immutable Git Tag:** `assignment-2.2`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_2_2.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_2_2.zip.sha256`

5.2 Verification Commands

```
# Run automated end-to-end socket demonstration
python demos/demo_assignment_2_2.py --mode automated

# Test multi-terminal mode (Terminal 1: Server, Terminal 2: Client)
# Terminal 1: python demos/demo_assignment_2_2.py --mode server
# Terminal 2: python demos/demo_assignment_2_2.py --mode client

# Run transport test suite
pytest -v tests/unit/test_transport.py tests/unit/test_message.py
```